

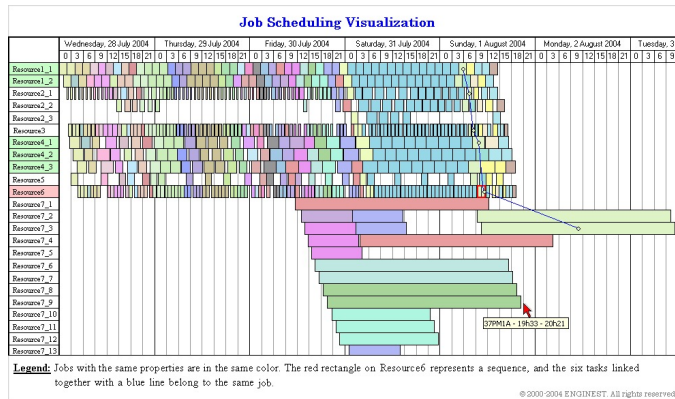
Programmation par contraintes dans des domaines finis

Alexander Bockmayr/Laurent Bougrain
LORIA / INRIA Nancy – Grand Est



Exemples de problèmes industriels

Ordonnancement de tâches



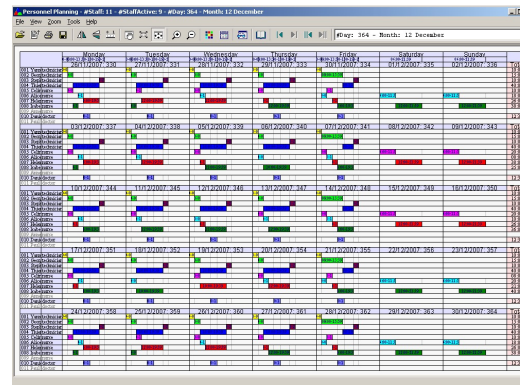
Objectifs :

- Diminution des ressources
- Diminution du temps d'exécution...

Contraintes :

- La tâche 1 nécessite la machine B pendant 24mn et doit être effectuée avant la tâche 7 et 12...

Emploi du temps



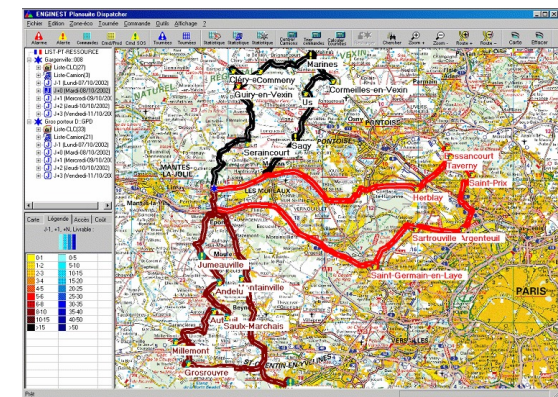
Objectifs :

- Diminuer les temps d'attente
- Diminuer le nombre de salles...

Contraintes :

- 1 technicien et 1 infirmier 24h/24
- 1 médecin en journée
- Réglementation social...

Tournées de véhicules



Objectifs :

- Diminution du kilométrage
- Diminution du nombre de véhicules
- Diminution du temps de travail...

Contraintes :

- Capacité des camions
- Disponibilité des véhicules...

Bibliographie

- BOCKMAYR Alexander et HOOKER John : Constraint programming, in K. Aardal, G. Nemhauser and R. Weismantel, eds, Handbooks in OR & MS, vol. 12, Ed. Elsevier, 2004
- MACKWORTH Alan : Consistency in networks of relation, Artificial Intelligence, Vol 8, pp99-118, North-Holland Publishing compagny, 1977
- MACKWORTH Alan : Constraint satisfaction, S. Shapiro Ed. in Encyclopedia of artificial intelligence, pp285-293, Wiley, 1992
- SIMONIS Helmut, Constraint Logic Programming, 1995
- SOLNON Christine : cours de Programmation par Contraintes, 2003
<http://www710.univ-lyon1.fr/~csolnon/Site-PPC/e-miage-ppc-som.htm>

Contraintes dans le domaine fini

Un problème de satisfaction de contraintes ou CSP (Constraint Satisfaction Problem) c'est :

- n variables $x_1, x_2, \dots, x_n$
- Pour chaque variable x_j , un domaine fini D_j de valeurs possibles (souvent $D_j = N$)
- m contraintes $C_1, C_2, \dots, C_m$ où $C_i \subseteq D_{i1} \times D_{i2} \times \dots \times D_{ik_i}$ est une relation entre k_i variables
- Une solution est l'affectation d'une valeur de D_j à x_j , pour tout $j=1 \dots n$ telle que toutes les relations C_i sont satisfaites

Programmation par contraintes

- Idée de base : C'est une programmation qui inclue des contraintes, c'est-à-dire que le langage de programmation possède un solveur de contraintes.
- Contraintes : linéaire, non linéaire, orientée objet, booléenne...
- Programmation : logique, fonctionnelle, orientée objet, impérative, concurrente...
- Systèmes : Prolog III/IV, CHIP, ECLiPSe, ILOG, gprolog, Choco...

Un problème de cryptarithmétique

Soit l'addition suivante :

```
  SEND
+ MORE
-----
 MONEY
```

où chaque lettre représente un chiffre différent (compris entre 0 et 9).

On souhaite connaître la valeur de chaque lettre, sachant que la première lettre de chaque mot représente un chiffre différent de 0.

1. Modéliser ce problème sous la forme d'un CSP.

Un problème de cryptarithmétique : Solution 1

1. Modéliser ce problème sous la forme d'un CSP.

- Soit $X = \{S, E, N, D, M, O, R, Y\}$ la liste des variables
- Soient $D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y$ les domaines respectivement de S, E, N, D, M, O, R, Y .

On a:

$$D_S = D_M = \{1, \dots, 9\}$$

$$D_E = D_N = D_D = D_O, D_R, D_Y = \{0, 1, \dots, 9\}$$

- Soient les contraintes suivantes:

$$C_1 : 1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E = 10000*M + 1000*O + 100*N + 10*E + Y$$

$$C_2 : S \neq E$$

$$C_3 : S \neq N$$

$$C_4 : S \neq D$$

...

$$C_{29} : R \neq Y$$

Un problème de cryptarithmétique : Solution 2

1. Modéliser ce problème sous la forme d'un CSP.

- Soit $X = \{S, E, N, D, M, O, R, Y, R1, R2, R3\}$ la liste des variables
- Soient $D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R1}, D_{R2}, D_{R3}$ les domaines respectivement de $S, E, N, D, M, O, R, Y, R1, R2, R3$.

On a:

$$D_S = D_M = \{1, \dots, 9\}$$

$$D_E = D_N = D_D = D_O, D_R, D_Y = \{0, 1, \dots, 9\}$$

$$D_{R1}, D_{R2}, D_{R3} = \{0, 1\}$$

- Soient les contraintes suivantes:

$$C_1 : D + E = Y + 10 \cdot R1$$

$$C_2 : R1 + N + R = E + 10 \cdot R2$$

$$C_3 : R2 + E + O = N + 10 \cdot R3$$

$$C_4 : R3 + S + M = O + 10 \cdot M$$

$$C_5 : S \neq E$$

...

$$C_{32} : R \neq Y$$

Le problème du zèbre

On s'intéresse au problème suivant, posé initialement par Lewis Carroll : *Cinq maisons consécutives, de couleurs différentes, sont habitées par des hommes de différentes nationalités. Chacun possède un animal différent, a une boisson préférée différente et fume des cigarettes différentes. De plus, on sait que :*

- *Le norvégien habite la première maison,*
- *La maison à coté de celle du norvégien est bleue,*
- *L'habitant de la troisième maison boit du lait,*
- *L'anglais habite la maison rouge,*
- *L'habitant de la maison verte boit du café,*
- *L'habitant de la maison jaune fume des kools,*
- *La maison blanche se trouve juste après la verte,*
- *L'espagnol a un chien,*
- *L'ukrainien boit du thé,*
- *Le japonais fume des cravens,*
- *Le fumeur de old golds a un escargot,*
- *Le fumeur de gitanes boit du vin,*
- *Le voisin du fumeur de Chesterfields a un renard,*
- *Le voisin du fumeur de kools a un cheval.*

1. Modéliser ce problème sous la forme d'un CSP.
2. *Qui boit de l'eau ?*
3. *A qui appartient le zèbre ?*

Le problème du zèbre : solution

- On définit le CSP (X,D,C) tel que
- Variables du problème: on associe une variable par attribut (couleur, animal, boisson, nationalité, cigarette) $X = \{\text{blanche, rouge, verte, jaune, bleue, norvégien, anglais, ukrainien, japonais, espagnol, cheval, renard, zèbre, escargot, chien, thé, eau, lait, café, vin, kools, chesterfields, old_golds, cravens, gitanes}\}$
- Domaines des variables: $D(X_i) = \{1,2,3,4,5\}$, pour toute variable X_i de X
- Contraintes:
 - On pose tout d'abord une contrainte pour chaque assertion de l'énoncé : $\text{norvégien} = 1$,
 $\text{bleue} = (\text{norvégien} + 1) \text{ ou } (\text{norvégien} - 1)$
 $\text{lait} = 3$,
 $\text{anglais} = \text{rouge}$,
 $\text{verte} = \text{café}$,
 $\text{jaune} = \text{kools}$,
 $\text{blanche} = \text{verte} + 1$,
 $\text{espagnol} = \text{chien}$,
 $\text{ukrainien} = \text{thé}$,
 $\text{japonais} = \text{cravens}$,
 $\text{old_golds} = \text{escargot}$,
 $\text{gitanes} = \text{vin}$,
 $(\text{chesterfields} = \text{renard} + 1) \text{ ou } (\text{chesterfields} = \text{renard} - 1)$,
 $(\text{kools} = \text{cheval} + 1) \text{ ou } (\text{kools} = \text{cheval} - 1)$
 - De plus, toutes les variables de même "type" doivent avoir des valeurs différentes (il ne peut pas y avoir plusieurs maisons qui ont la même couleur, ou un même animal, ...) $\text{blanche} \neq \text{rouge} \neq \text{verte} \neq \text{jaune} \neq \text{bleue}$,
 $\text{thé} \neq \text{eau} \neq \text{lait} \neq \text{café} \neq \text{vin}$,
 $\text{norvégien} \neq \text{anglais} \neq \text{ukrainien} \neq \text{japonais} \neq \text{espagnol}$,
 $\text{cheval} \neq \text{renard} \neq \text{zèbre} \neq \text{escargot} \neq \text{chien} \neq \text{thé}$,
 $\text{kools} \neq \text{chesterfields} \neq \text{old_golds} \neq \text{cravens} \neq \text{gitanes}$

Problème d'affectation parcimonieuse

- Une école souhaite affecter ses étudiants à des stages en entreprise en respectant au mieux le choix des étudiants et des entreprises. Elle demande pour cela d'une part à chacun des étudiants de classer par ordre de préférence les stages qui les intéressent et d'autre part, aux entreprises de classer par ordre de préférence les étudiants qui les intéressent. Pour tenir compte au mieux des désirs des uns et des autres, ces listes peuvent être incomplètes. Par ailleurs, dans le cas où un étudiant indécis n'arrive pas à trancher entre plusieurs stages qui lui semblent également intéressants, il peut les classer ex aequo (idem pour les entreprises). Pour simplifier, on supposera que l'on a autant d'étudiants que d'entreprises. Il s'agit alors, à partir de ces listes de préférences éventuellement incomplètes et avec des ex aequo, de former des affectations stables. Par stable, on entend que personne ne refuse l'affectation proposée sous prétexte qu'on aurait pu faire mieux. Par exemple, si France est affectée à Philips alors qu'elle préférerait France Télécom et si dans le même temps Philippe est affecté à France Télécom alors qu'il préférerait Philips, nous sommes en présence d'une instabilité

Problème d'affectation parcimonieuse

- Les classements des étudiants sont les suivants :
 - E1 préfère B2, puis (B6 et B4 ex aequo) ;
 - E2 préfère (B2, B5), B6 ;
 - E3 préfère B1, B3, B6 ;
 - E4 préfère B6, B3 ;
 - E5 préfère B2, B1, B5 ;
 - E6 préfère B6, (B4, B2), B5, B1.
- Les classements des entreprises sont les suivants :
 - B1 préfère (E5, E3), E6 ;
 - B2 préfère E2, E5, E1, E6 ;
 - B3 préfère (E3, E4) ; B4 préfère E6, E1 ;
 - B5 préfère E5, E2, E6 ;
 - B6 préfère E1, (E4, E6), E2, E3.

Q. 1 : Modéliser ce problème sous la forme d'un problème de satisfaction de contraintes (variables, domaines, contraintes).

Problème d'affectation parcimonieuse : solution

- Variables

$X = \{\text{boite-de-1}, \text{boite-de-2}, \text{boite-de-3}, \text{boite-de-4}, \text{boite-de-5}, \text{boite-de-6}\}$

- Domaines :

Pour chaque variable $\text{boite-de-}i$, le domaine associé contient l'ensemble des entreprises qui sont classées par i et qui ont classées i :

- $D(\text{boite-de-1}) = \{B2, B4, B6\}$
- $D(\text{boite-de-2}) = \{B2, B5, B6\}$
- $D(\text{boite-de-3}) = \{B1, B3, B6\}$
- $D(\text{boite-de-4}) = \{B3, B6\}$
- $D(\text{boite-de-5}) = \{B1, B2, B5\}$
- $D(\text{boite-de-6}) = \{B1, B2, B4, B5, B6\}$

Il n'y a pas de problème particulier : toutes les entreprises choisies par les étudiants ont sélectionnées ces étudiants.

- Contraintes

- $C1 = \forall i \in \{1..6\} \forall j \in \{1..6\}, j \neq i, \text{boite-de-}i \neq \text{boite-de-}j$
- $C2 = \forall i \in \{1..6\} \forall j \in \{1..6\}, j \neq i, \text{non}(\text{préfère}(i, \text{boite-de-}j, \text{boite-de-}i)) \text{ et } \text{préfère}(\text{boite-de-}j, i, j))$

Résolution par génère et teste

- Instancier les variables (dans l'ordre de déclaration à défaut d'heuristique de sélection).
- Quand toutes les variables sont instanciées, déterminer si toutes les contraintes sont respectées.
- Si une contrainte n'est pas satisfaite, retourner à la dernière variable dont le domaine contient encore des valeurs non affectées.

Résolution par backtracking

- Instancier les variables dans l'ordre.
- Dès que toutes les variables d'une contrainte sont instanciées, déterminer si la contrainte est respectée.
- Si la contrainte n'est pas satisfaite, retourner à la dernière variable dont le domaine contient encore des valeurs non affectées, sinon continuer l'instanciation.

Problèmes d'efficacité

Mackworth 77

- Si le domaine D_i de la variable x_i contient une valeur v qui ne satisfait pas C_i , alors toutes les instanciations futures aboutiront à des échecs.
- Si nous instancions les variables dans l'ordre $x_1, x_2, \dots, x_n$ et si pour $x_i = v$ il n'y a pas de valeur $w \in D_j$, pour $j > i$, telle que $C_{ij}(v, w)$ soit satisfaite, alors le backtracking essaiera toutes les valeurs de x_i échouera et essaiera toutes les valeurs de x_{i-1} puis toutes les valeurs de x_i de nouveau, échouera et ce jusqu'à ce qu'on essaie toutes les combinaisons de $x_{i+1}, x_{i+2}, \dots, x_j$ avant de découvrir finalement que la valeur v n'est pas possible pour x_i .

Consistance locale

- Soit un CSP avec uniquement des contraintes unaires et binaires.
- Soit G le graphe des contraintes
 - Chaque variable x_i est représentée par un nœud i
 - Chaque paire de variable x_i, x_j apparaissant dans une même contrainte est représentée par deux arcs (i,j) et (j,i) .
- Le nœud i est consistant si $C_i(v)$ est satisfait pour tout $v \in D_i$
- L'arc (i,j) est consistant si pour tout $v \in D_i$ satisfaisant $C_i(v)$ il existe $w \in D_j$ satisfaisant $C_j(w)$ telles que $C_{ij}(v,w)$ soit satisfaite.

Remarque : le fait que l'arc (i,j) soit consistant n'implique pas nécessairement que l'arc (j,i) le soit également
- Le graphe des contraintes est consistant au niveau des nœuds (resp. des arcs) si tous les nœuds (resp. les arcs) sont consistants.

L'algorithme Arc-Consistency [Mackworth, 77]

L'algorithme Arc-Consistency (AC-3)

pour i de 1 à n faire $D_i \leftarrow \{v \in D_i \mid C_i(v)\}$

$Q \leftarrow \{(i,j) \mid (i,j) \in \text{arcs}(G)\}$

tant que Q n'est pas vide faire

 sélectionner et supprimer un arc (i,j) de Q

 si $\text{REVISE}(i,j)$ est vraie alors $Q \leftarrow Q \cup \{(k,i) \mid (k,i) \in \text{arcs}(G), k \neq i, k \neq j\}$

Procédure $\text{REVISE}(i,j)$

$\text{DELETE} \leftarrow \text{Faux}$

 pour toute valeur $v \in D_i$ faire

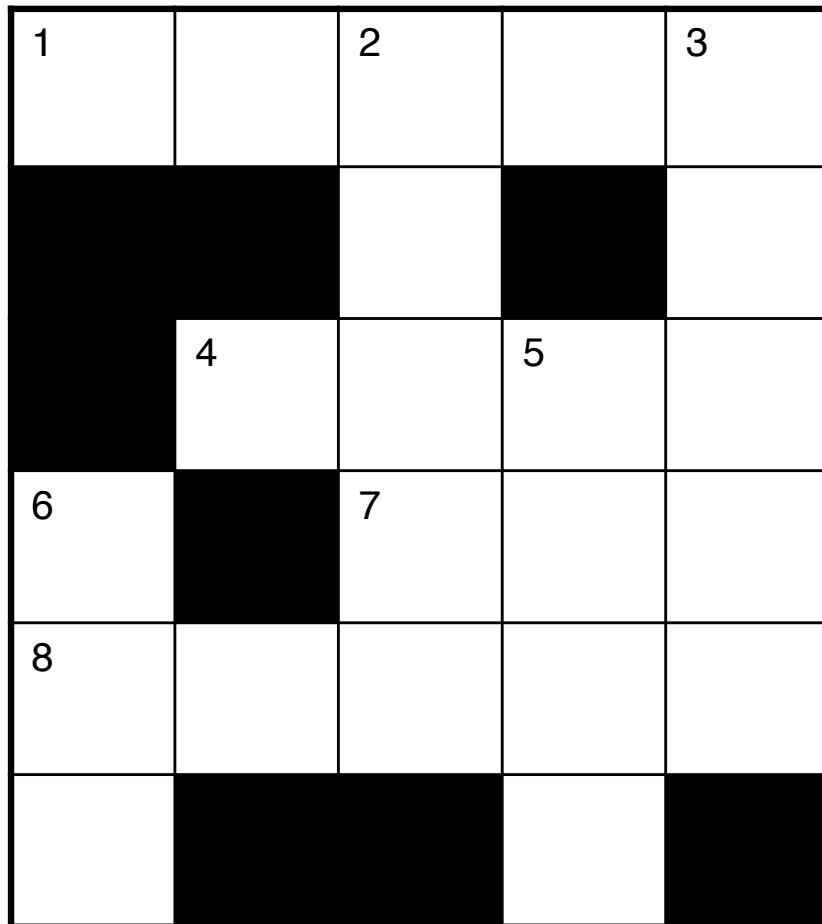
 s'il n'y a pas de valeur $w \in D_j$ telle que $C_{ij}(v,w)$ soit satisfaite alors

 supprimer v de D_i

$\text{DELETE} \leftarrow \text{Vrai}$

 retourner DELETE

Le problème de mots placés



Liste de mots

Aft
Ale
Eel
Heel
Hike
Hoses
Keel
Knot
Laser
Lee
Line
Sails
Sheet
Steer
Tie

1. Faire le graphe G des contraintes
2. Appliquer l'algorithme AC-3

Les mots-placés : modélisation

- Chaque mot cherché est associé à une variable x_i
 - Le domaine de définition de chaque variable est :
 $D_i = \{\text{Aft, Ale, Eel, Heel, Hike, Hoses, Keel, Knot, Laser, Lee, Line, Sails, Sheet, Steer, Tie}\}$
 - On note C_i la contrainte unaire de chaque variable x_i : $\text{longueur}(x_1)=5, \dots$
 - On note C_{ij} les contraintes binaires : $C_{12}=C(x_1, x_2)$ $\text{lettre}(x_1, 3)=\text{lettre}(x_2, 1), \dots$
1. Faire le graphe G des contraintes

Les mots placés : application de l'algorithme AC3

2. Appliquer l'algorithme AC-3

$D_1=D_2=D_3= \{\text{Hoses, Laser, Sails, Sheet, Steer}\}$

$D_4=D_5= \{\text{Heel, Hike, Keel, Knot, Line}\}$

$D_6=D_7= \{\text{Aft, Ale, Eel, Lee, Tie}\}$

$D_8= \{\text{Hoses, Laser, Sails, Sheet, Steer}\}$

$Q=\{(1,2),(2,1),(1,3),(3,1),(2,4),(4,2),(2,7),(7,2),(2,8),(8,2),(3,4),(4,3),(3,7),(7,3),(3,8),(8,3),$
 $(4,5),(5,4),(5,7),(7,5),(5,8),(8,5),(6,8),(8,6)\}$

sélectionner (1,2),

$Q=\{(\textcolor{red}{1},\textcolor{red}{2}), (2,1), (1,3), (3,1), (2,4), (4,2), (2,7), (7,2), (2,8), (8,2), (3,4), (4,3), (3,7), (7,3), (3,8), (8,3),$
 $(4,5), (5,4), (5,7), (7,5), (5,8), (8,5), (6,8), (8,6)\}$

REVISE (1,2) <- true ($D_1= \{\text{Hoses, Laser}\}$)

$Q=\{(2,1),(1,3),(\textcolor{teal}{3},\textcolor{teal}{1}), (2,4), (4,2), (2,7), (7,2), (2,8), (8,2), (3,4), (4,3), (3,7), (7,3), (3,8), (8,3), (4,5), (5,4),$
 $, (5,7), (7,5), (5,8), (8,5), (6,8), (8,6)\}$

Anticipations

Appliquer dynamiquement la consistance locale pendant la recherche.

- Forward checking : après l'affectation de la valeur v à x , éliminer pour toutes les variables y non encore instanciées les valeurs de D_y qui sont incompatibles avec v .
- Partial lookahead : établir la consistance des arcs pour tout couple (y, y') de variables non encore instanciées et où y sera instanciée avant y' .
- Full lookahead : établir la consistance des arcs pour tout couple (y, y') de variables non encore instanciées.

Les 8 reines

- Chaque reine doit se trouver sur une ligne différente.
Soit x_i la colonne de la reine qui se trouve sur la ligne i , $i = 1, \dots, 8$
- Chaque domaine vaut $D_i = \{1, \dots, 8\}$
- Les contraintes binaires sont C_{ij} :
 $x_i \neq x_j$, pour $1 \leq i < j \leq 8$ (colonne)
 $x_i \neq x_j + (j - i)$, pour $1 \leq i < j \leq 8$ (diagonale 1)
 $x_i \neq x_j - (j - i)$, pour $1 \leq i < j \leq 8$ (diagonale 2)

Les 8 reines (Forward checking)

Q							
X	X	Q					
X	X	X	X	Q			
X	Q	X	X	X	X		
X	X	X		X	X	X	
X	X	X	X	X	X	X	X
X	X	X		X		X	X
X	X	X		X	X		X

$X_1=1$

$X_2=3$

$X_3=5$

$X_4=2, 7$ (plus de valeurs pour X_6)

...

Les 8 reines (Partial Lookahead)

Q							
X	X	Q					
X	X	X	X	Q			
X	●	X	X	X	X		
X		X	●	X	X	X	
X	X	X	↓	X	X	X	X
X		X		X		X	X
X		X		X			X

$X_1=1$

$X_2=3$

$X_3=5$ ($D_4 = D_4 \setminus \{2\}$, $D_5 = D_5 \setminus \{4\}$)

$X_4=7$

Les 8 reines (Full Lookahead)

Q							
X	X	Q					
X	X	X	X	Q			
X	●	X	X	X	X	↖	↗
X	↖	X	●	X	X	X	●
X	X	X	↖	X	X	X	X
X	●	X	●	X	↖	X	X
X	●	X	●	X	●	●	X

$X_1=1$

$X_2=3$

$X_3=5$ ($D_4 = D_4 \setminus \{2\}$, $D_5 = D_5 \setminus \{4\} \dots$)

$X_3=6$

Struture classique d'un programme de satisfaction

- Déclaration des variables et de leur domaines de définitions
- Définition des contraintes
- Énumération (labeling)

Les solveurs de contraintes réalisent uniquement les consistances locales.

Dans le but d'obtenir les consistances globales, les domaines doivent être énumérés.

Problème de cryptarithmétique

- SEND+MORE = MONEY
- Attribuer à chaque lettre de {S,E,N,D,M,O,R,Y} un chiffre différent entre 0 et 9 tel que l'équation soit satisfaite.

```
Top([S,E,N,D,M,O,R,Y]) :-  
  [S,E,N,D,M,O,R,Y]::0..9,  
  alldifferent ([S,E,N,D,M,O,R,Y]),  
  S #\= 0,  
  M #\= 0,  
  Send = 1000*S + 100*E + 10*N + D,  
  More = 1000*M + 100*O + 10*R + E,  
  Money = 10000*M + 1000*O + 100*N + 10*E + Y,  
  Send + More #= Money,  
  labeling ([S,E,N,D,M,O,R,Y]).
```

?- top(X)

X = [9,5,6,7,1,0,8,2]

Par propagation de contraintes, l'espace de recherche est réduit à $S = 9$, $M = 1$, $O = 0$, $E : [4..7]$, $N : [5..8]$, $R, D, Y : [2..8]$ au noeud racine.

Un seul backtracking est nécessaire pour trouver une solution.

Stratégies de sélection dynamique des variables

Par défaut, l'ordre d'instanciation des variables est celui de déclaration.
Des heuristiques peuvent permettre de diminuer l'arbre de recherche.

Nom	Stratégie
First_fail	variable avec le plus petit domaine [au moment de l'instanciation].
Most_constrained	variable qui apparaît dans le plus de contraintes et qui possède le plus petit domaine
Smallest (largest)	variable qui possède la valeur la plus petite (la plus grande)
Max_regret	variable qui a le plus grand écart entre sa valeur la plus petite et la suivante.

Heuristiques d'énumération des valeurs

- **indomain(X)** : sélectionne les valeurs pour les variables X, en commençant par les plus petites
- **indomain(X, method)** : sélectionne les valeurs pour les variables X, en utilisant une des méthodes suivante : {min, max, middle}
- **Labeling(List,0,Method, Pred)** : Affecte des valeurs aux variables de la list List en utilisant
 - Method pour choisir la variable
 - Pred pour choisir la valeur

Exemple : labeling(List, 0, first_fail, indomain).

- **min_max(Goal,C)** : trouve une solution de Goal qui minimise le coût (maximal) de C.

Exemple : min_max(labeling(List,0,first_fail,indomain),C).

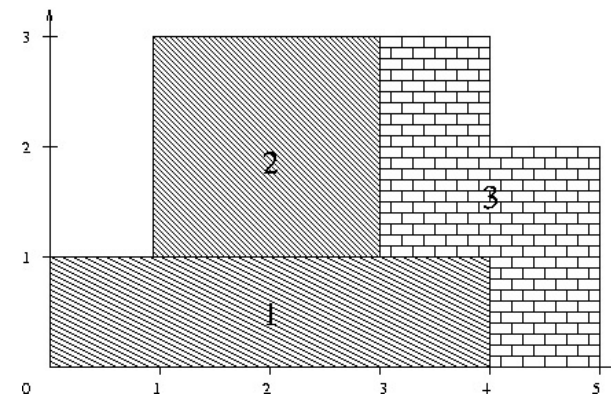
Contraintes globales

- **alldifferent**($[X_1, \dots, X_n]$) : toutes les variables doivent prendre des valeurs différentes
i.e. $X_1 \neq \dots \neq X_n$

Exemples: cryptoarithmétique, voyageur de commerce...

- **cumulative**($[S_1, \dots, S_n], [D_1, \dots, D_n], [R_1, \dots, R_n], L$) avec
 - n le nombre de tâches,
 - S_i l'heure de démarrage de la tâche i ,
 - D_i la durée de la tâche i ,
 - R_i le nombre de ressources demandées par la tâche i à chaque instant
 - L le nombre maximum de ressources disponibles à un instant donné

Exemples: **cumulative**($[0,3,1], [4,2,2], [1,2,2], 3$)



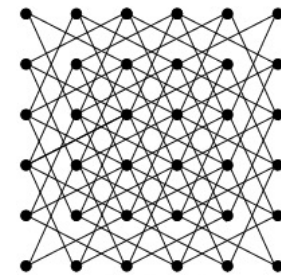
Contraintes globales

- **diffn**([[$O_{11}, \dots, O_{1n}, L_{11}, \dots, L_{1n}$], ..., [$O_{m1}, \dots, O_{mn}, L_{m1}, \dots, L_{mn}$]]) avec
 - m le nombre d'objets rectangulaires,
 - n la dimension des objets
 - O_{ij} position d'origine du $i^{\text{ème}}$ objet dans la $j^{\text{ème}}$ dimension
 - L_{ij} taille du $i^{\text{ème}}$ objet dans la $j^{\text{ème}}$ dimension

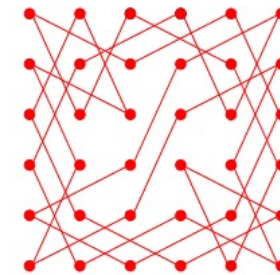
place sans recouvrement m objets rectangulaires i de dimension n de tailles $L_{i1}, \dots, L_{in}$

- **cycle**(N,[$S_1, \dots, S_m$]) avec
 - N le nombre de cycles
 - S_i , les cycles
 - m le nombre de nœuds du graphe

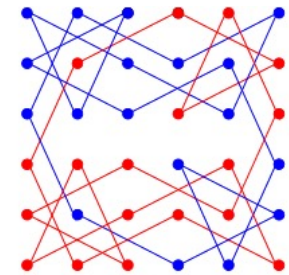
Génère le recouvrement d'un graphe par N cycles ou circuits. Chaque nœud n'appartient qu'à un cycle.



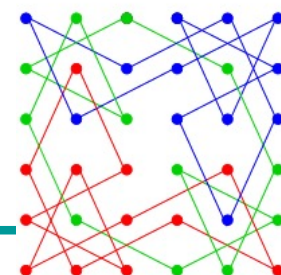
Graph of potential moves of a 6 X 6 chessboard



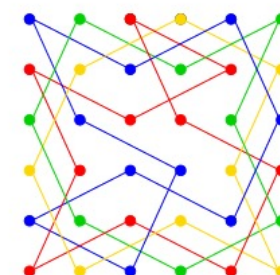
1 knight
(36 moves)



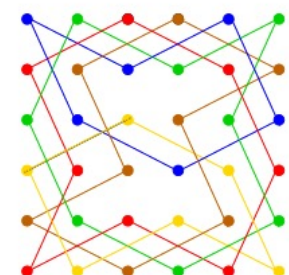
2 knights
(18 and 18 moves)



3 knights
(12, 12 and 12 moves)



4 knights
(8, 8, 10 and 10 moves)



5 knights
(6, 6, 8, 8 and 8 moves)

Applications

Remplissage de conteneurs (bin packing problem)

- Répartir n objets de taille L_i avec $i=1,\dots,n$ dans au plus m conteneurs de capacité c .
- optimisation : Trouver le nombre minimal de conteneurs nécessaires.

Placement

- Placer sans recouvrement m objets rectangulaires i de dimension n de tailles $L_{i1},\dots,L_{in}$

Tournée de véhicules (vehicle routing problem)

- Affecter chaque point de passage à un véhicule parmi n et préciser pour chaque véhicule l'ordre de visite.

Systèmes de propagation de contraintes [Bockmayer, Hooker]

Système	Disponibilité	Contraintes	Langage	Site web
B-prolog	Commercial	Domaine fini	Prolog	http://www.picat-lang.org/bprolog/
CHIP	Commercial	Domaine fini, hybride, booléen, rationnel linéaire	C, C++	www.cosytec.com
Choco	Gratuit	Domaine fini	Claire	https://choco-solver.org
Eclipse	Gratuit si but non lucratif	Domaine fini, hybride	Prolog	eclipseclp.org/
GNU prolog	Gratuit	Domaine fini	Prolog	www.gprolog.org/
ILOG CPLEX	Commercial	Domaine fini, hybride	C++, Java, .net, python	https://www.ibm.com/fr-fr/products/ilog-cplex-optimization-studio
NCL	Commercial	domaine fini	Java	https://www.cs.cmu.edu/Groups/AI/lang/prolog/impl/parallel/ncl/0.html
Mozart	Gratuit	Domaine fini	Oz	mozart.github.io
Prolog IV	Commercial	Domaine fini, arithmétique linéaire et non linéaire par intervalles	Prolog	prolog-heritage.org/
Sicstus	Commercial	Domaine fini, booléen, réel/rationnel linéaire	Prolog	sicstus.sics.se